

1. What is Data Engineering?

Data Engineering is the process of **collecting, moving, storing, transforming, and preparing data** so that it can be reliably used for:

- Analytics
- Business Intelligence
- Reporting
- Machine Learning
- Business decision-making

A Data Engineer primarily builds and maintains the systems and pipelines that make this possible.

Basic Data Engineering Flow

Source Systems



Data Ingestion



Data Storage



Data Transformation



Processed / Business Data



Analytics / BI / ML

This flow is the foundation of most Data Engineering systems.

2. Data Sources

Every data pipeline starts with a **source system**.

Common sources include:

Application Databases

APIs

CSV / JSON Files

SaaS Applications

Logs

Streaming Systems

Example: E-commerce Company

Orders → PostgreSQL

Customers → MySQL

Products → API

Transactions → Database

Events → Kafka

Before designing a pipeline, a Data Engineer should understand:

- Where is the data coming from?
 - What format is it in?
 - How much data is generated?
 - How frequently does it change?
 - How quickly does the business need it?
-

3. Data Ingestion

Data Ingestion means moving data from source systems into the data platform.

Example:

PostgreSQL



Ingestion Pipeline



Cloud Storage

The ingestion process may run periodically or continuously depending on the business requirement.

4. Batch vs Streaming

These are two common ways of processing data.

Batch Processing

Data is collected and processed at scheduled intervals.

Examples:

Every 1 hour

Every night

Once every day

Example:

A company processes all of yesterday's orders every morning.

Suitable For

- Daily reports
 - Historical processing
 - Periodic ETL pipelines
-

Streaming

Data is processed continuously or near real time as events arrive.

Example:

Payment Event



Streaming Pipeline



Fraud Detection

Suitable For

- Fraud detection
- Live monitoring
- Real-time analytics
- Application events

Remember

Batch → Process periodically

Streaming → Process continuously / near real time

Not every pipeline requires streaming. The business latency requirement should determine the approach.

5. Full Load vs Incremental Load

Suppose a source table contains:

100 million records

but only:

50,000 records

change every day.

Processing all 100 million records daily may be unnecessary.

Full Load

The complete dataset is processed again.

Read Everything



Process Everything



Load Everything

Commonly used for:

- Initial loads
- Small datasets
- Complete rebuilds

Incremental Load

Only new or changed records are processed.

New / Updated Records



Process



Update Target

Changes can be identified using:

- Updated timestamp
- Watermark
- CDC (Change Data Capture)

Why Incremental Processing?

- Faster processing
 - Lower compute usage
 - Reduced cost
 - Better scalability
-

6. ETL vs ELT

ETL — Extract, Transform, Load

Source



Extract



Transform



Load

Data is transformed before being loaded into the final target system.

ELT — Extract, Load, Transform

Source



Extract



Load



Transform

Raw data is loaded first and transformations are performed inside the target platform.

Key Difference

The important question is:

Where does the transformation happen?

Modern cloud data warehouses and lakehouse platforms commonly support ELT because they provide scalable compute for transformations.

7. Data Lake vs Data Warehouse

Data Lake

A Data Lake provides scalable storage for large amounts of raw and processed data.

It can contain formats such as:

CSV
JSON
Parquet
Logs

Examples of commonly used storage:

- Amazon S3
 - Azure Data Lake Storage
 - Google Cloud Storage
-

Data Warehouse

A Data Warehouse is designed primarily for **structured analytical data and reporting**.

Typical use cases:

- BI dashboards

- Business reports
- Analytical queries
- Aggregations

Examples:

- Snowflake
- BigQuery
- Amazon Redshift

Simple Difference

Data Lake

→ Flexible storage for raw and processed data

Data Warehouse

→ Structured data optimized for analytics

8. Data Transformation

Raw source data is rarely ready for business use.

A Data Engineer performs transformations such as:

- Removing duplicates
- Handling null values
- Correcting data types
- Standardizing formats
- Joining datasets
- Filtering invalid records
- Creating derived columns
- Aggregating data

Example

Source:

order_id
customer_id
amount
discount
status

Business may require:

`final_amount = amount - discount`

This transformation converts source data into information useful for downstream consumers.

9. Data Quality

A successful pipeline does not automatically mean the data is correct.

Data quality checks ensure that the data reaching downstream systems is reliable.

Common Checks

Is order_id NULL?

Are duplicate order IDs present?

Is amount negative?

Did today's source file arrive?

Does source count match target count?

Poor data quality can result in incorrect dashboards and business decisions.

A Data Engineer is responsible not only for **moving data**, but also for making sure it is **reliable**.

10. Data Orchestration

A pipeline normally contains multiple dependent tasks.

Example:

Ingest Data



Validate Data



Transform Data

↓
Load Final Tables
↓
Refresh Dashboard

These tasks need to run in the correct sequence.

Orchestration manages this workflow.

It typically handles:

- Scheduling
- Dependencies
- Retries
- Failure handling
- Monitoring

Common tools include:

Apache Airflow
Azure Data Factory
Databricks Workflows

11. End-to-End Data Engineering Flow

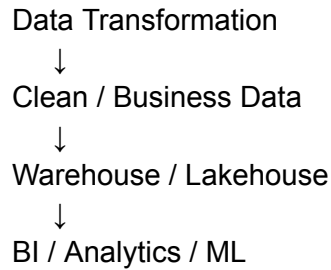
Consider an e-commerce company.

Sources

Orders Database
Customer Database
Product API
Website Events

Pipeline

Source Systems
↓
Data Ingestion
↓
Raw Storage / Data Lake
↓
Data Validation
↓



Around this pipeline, we also need:

Data Quality
Orchestration
Monitoring
Security

This represents the basic architecture of a production Data Engineering system.

12. Where Do Common Data Engineering Tools Fit?

Technology	Common Purpose
SQL	Querying and transforming data
Python	Automation, APIs and data processing
Apache Spark	Distributed data processing
Databricks	Data engineering and lakehouse workloads
Kafka	Event streaming
Airflow / ADF	Pipeline orchestration
Snowflake / BigQuery	Analytical data platforms
S3 / ADLS / GCS	Cloud data storage

The important principle is:

Learn the Data Engineering concept first and then understand which tool solves that problem.

Quick Revision

Before moving ahead, make sure you can explain these concepts in your own words:

Data Engineering Flow:

Source → Ingestion → Storage → Transformation → Serving → Consumption

Batch vs Streaming:

Periodic processing vs continuous/near-real-time processing.

Full vs Incremental:

Process everything vs process only new/changed data.

ETL vs ELT:

Transform before loading vs transform after loading.

Data Lake vs Warehouse:

Flexible data storage vs analytics-focused structured platform.

Data Quality:

Making sure data is complete, valid, consistent, and reliable.

Orchestration:

Managing pipeline scheduling, dependencies, retries, and failures.

Data with Anurag — DataX Bootcamp

If you want to learn Data Engineering through a structured path instead of learning disconnected tools, you can explore:

DataX Bootcamp – Zero to Job Ready AI-Enabled Data Engineer in 90 Days

The program covers the Data Engineering journey through practical learning, including **SQL, Python, Data Modeling, PySpark, Big Data fundamentals, Databricks, Cloud, hands-on projects, migration scenarios, and interview preparation.**

DataX Bootcamp:

[Explore DataX Bootcamp -: https://login.datawithanurag.com/services/dataxbootcamp](https://login.datawithanurag.com/services/dataxbootcamp)

[Coupon Code -: YT_FAM](#)

Bootcamp Queries & Support:

WhatsApp Data with Anurag Team -: <https://wa.me/918957284012>

Data with Anurag:

datawithanurag.com